

Université Badji Mokhtar Annaba
Faculté de l'ingéniorat
Département d'informatique
Master 1, Réseaux et sécurité Informatique

Factory

Un patron de création

Préparé par: MAA. Yahiouche Salima

Factory method, définition

- Define an interface for creating an object, but let subclasses decide which class to instantiate.
→ Factory Method lets a class defer instantiation to subclasses.
- Permet de séparer la création d'objets dérivant d'une classe mère de leur utilisation.
- De ce fait, on a alors la possibilité de créer plusieurs objets issue d'une même classe mère.

Un premier exemple d'utilisation.....

- Supposez que vous gérez un pizza shop, une des méthodes les plus importantes est celle qui fait la commande:

```
Pizza commanderPizza(){  
    Pizza pizza = new Pizza();  
    pizza.préparer();  
    pizza.cuire();  
    pizza.couper();  
    pizza.mettreEnBoite();  
    return pizza;}  

```

Plusieurs types de pizza...

- Mais vous allez vendre plusieurs types de pizza, d'où l'introduction du type; ce qui va donner:

```
Pizza commanderPizza(String type){  
    Pizza pizza;  
    If (type.equals("fromage"))  
    {Pizza = new FromagePizza();}  
    else If (type.equals("champignon"))  
    {Pizza = new champignonPizza();}  
    else If (type.equals("pepperoni"))  
    {Pizza = new pepperoniPizza();}  
    pizza.préparer();  
    pizza.cuire();  
    pizza.couper();  
    pizza.mettreEnBoite();  
    return pizza;}  
}
```

Modifier car la concurrence oblige...

- Ensuite vous réalisez que vos concurrents vendent de nouveaux types de pizza, par exemple la végétarienne, et vous remarquez aussi, que la pepperoni n'est pas trop sollicitée; vous allez donc changer de menu:

```
Pizza commanderPizza(String type){  
    Pizza pizza;if (type.equals("fromage"))  
    {Pizza = new FromagePizza();}  
    else If (type.equals("champignon"))  
    {Pizza = new champignonPizza();}  
    else If (type.equals("végétarienne"))  
    {Pizza = new végétariennePizza();}  
    pizza.préparer();  
    pizza.cuire();  
    pizza.couper();  
    pizza.mettreEnBoite();  
    return pizza;}
```

Discussion:

- —> le code précédant n'est pas fermé à la modification, si le pizza shop change ses offres de pizza, nous serons obligé de le modifier encore une fois;
- la partie de ce code qui varie souvent est celle qui concerne l'instanciation des classes concrètes: (FromagePizza, champignonPizza; végétarienne Pizza)..
- —> Solution : enlever le code de création des objets depuis la méthode commanderPizza et le déplacer vers un autre objet, qui sera lui concerné seulement par la création des différentes pizzas—>nous allons appeler cet Objet: un Factory

Factory: définition

- Un Factory s'occupe des détails de création d'un objet:
- à chaque fois que la méthode `commanderPizza` aura besoin d'une pizza, elle fera appel à l'objet `pizzaFactory` pour ce faire

La classe PizzaFactory

```
public class PizzaFactory {
```

```
// ce code est le même tiré de la méthode commandePizza, paramétré  
// avec le type de la pizza
```

```
public Pizza createPizza(String type){
```

```
    pizza=null;
```

```
    if (type.equals("fromage"))
```

```
    {pizza = new FromagePizza();}
```

```
    else if (type.equals("champignon"))
```

```
    {pizza = new champignonPizza();}
```

```
    else if (type.equals("végétarienne"))
```

```
    {pizza = new végétariennePizza();}
```

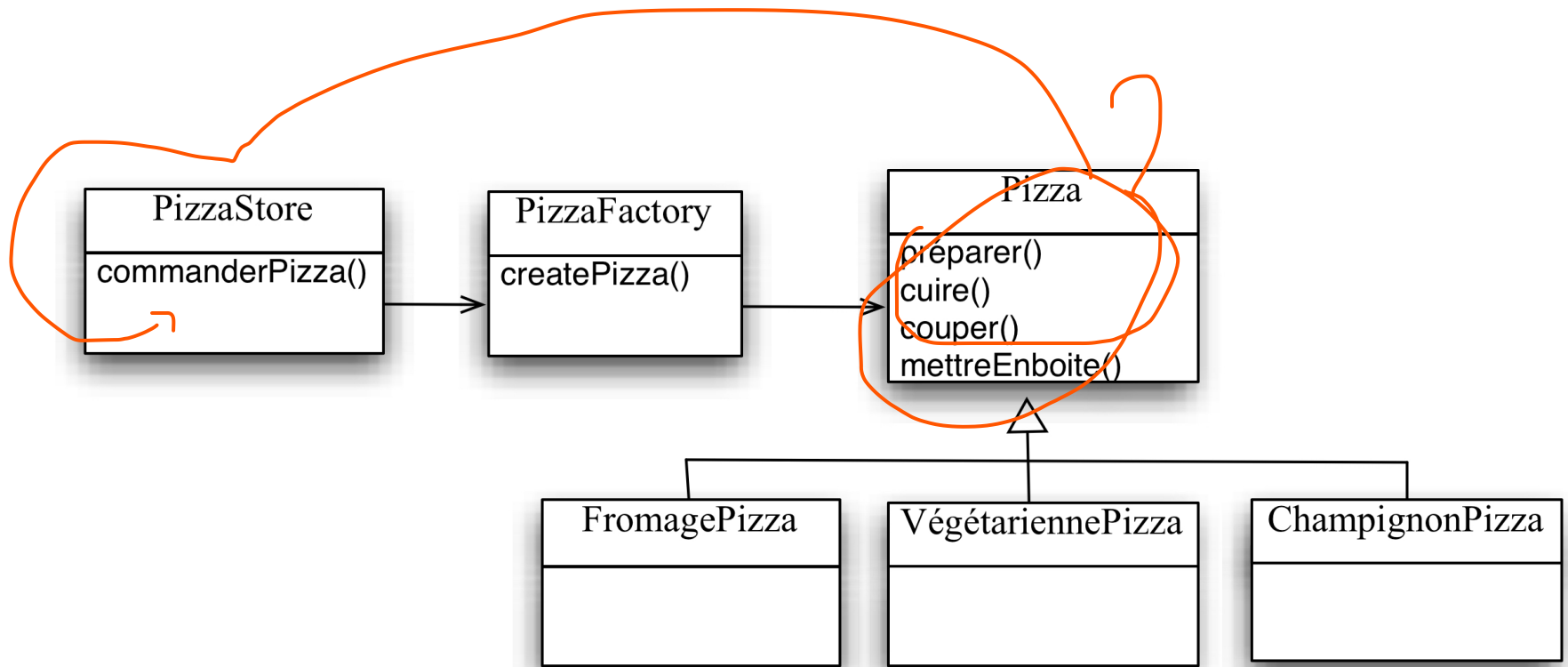
```
    return pizza;}}
```


La classe PizzaStore contenant la commande:

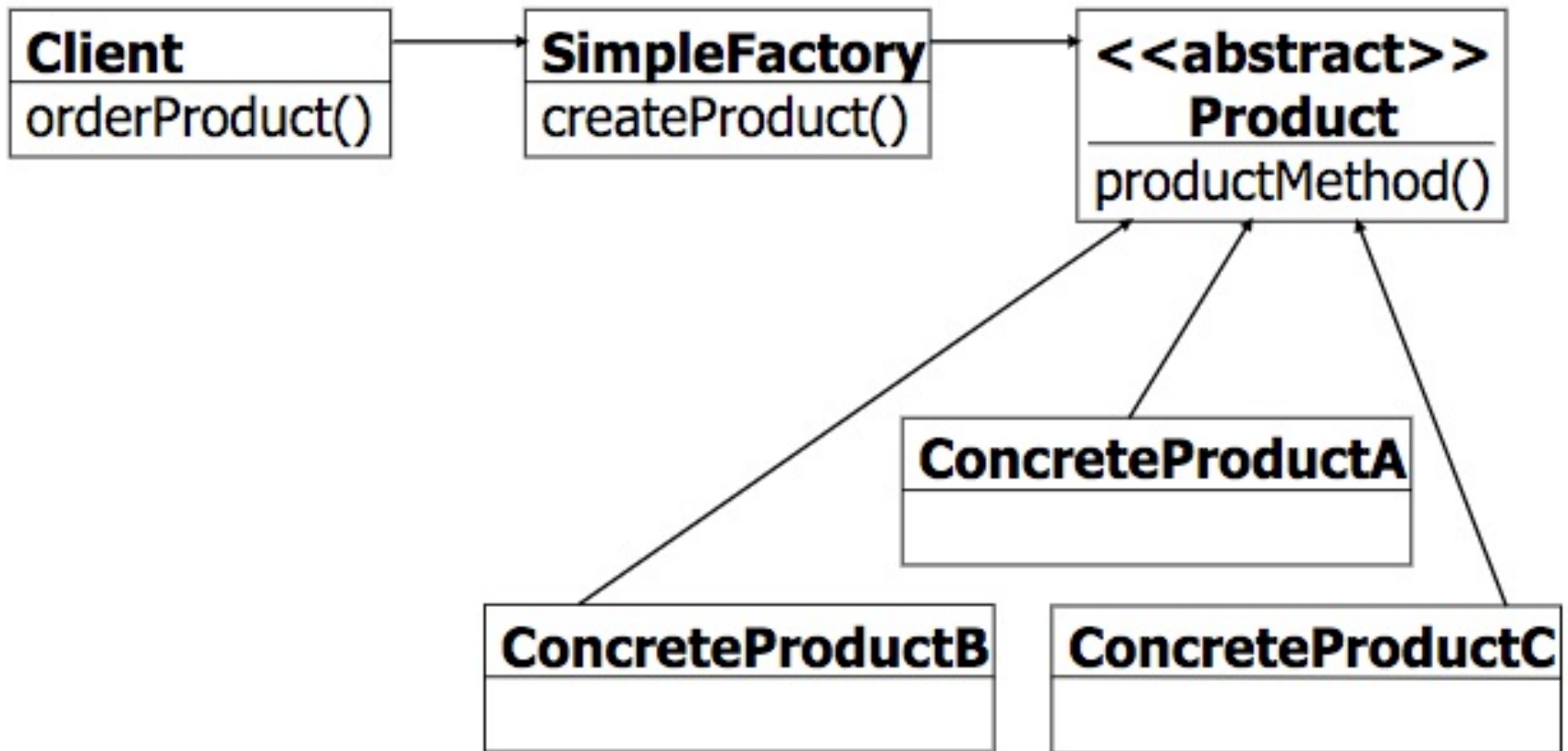
Revenons à la classe contenant la méthode CommanderPizza:

```
public class PizzaStore{  
    PizzaFactory factory;  
  
    public PizzaStore(PizzaFactory factory){  
        This.factory=factory;}  
  
    public Pizza commanderPizza (String type){  
        Pizza pizza;  
        pizza=factory.createPizza(type);  
        pizza.préparer();  
        pizza.cuire();  
        pizza.couper();  
        pizza.mettreEnBoite();  
        return pizza;}  
    // on peut mettre d'autres méthodes ici}
```

Diagramme correspondant aux classes précédentes



Simple Factory Pattern



Exemple d'instanciation

- Si par exemple on voudrait commander une pizza Fromage, dans le main d'une classe on fera:

```
PizzaFactory pizzaFactory= new PizzaFactory();
```

```
PizzaStore pizzaStore= new PizzaStore(pizzaFactory);
```

```
pizzaStore.commander(« Fromage »);
```

Le vrai Factory

- En réalité, ce que nous avons fait pour le pizza shop ne présente pas un vrai Pattern, il s'appelle: simple Factory;
- Nous aurons besoin du vrai Pattern “Factory method”, si nous voulons exporter notre savoir faire des pizza vers d'autres wilayas, ou pourquoi pas d'autres pays!!!??!, c'est ce que nous verrons dans le td

Voyons un autre exemple.....

- Votre système travaille avec une base de donnée Oracle, et un jour , votre responsable vous demande d'intégrer, deux nouvelles BDDs: SqlServer et MySql
- Vous serez donc obligé de modifier partout dans votre code , Là où il y a une connexion à la BDD Oracle
- → le code doit être dupliqué dans plusieurs endroits

Solution...

- Il faut **factoriser** le code de toutes les connexions dans une méthode:
- Principe OOP: il faut séparer la partie changeante du reste du code
- → créer l'objet Factory
- Dans notre cas, il aura pour role, la création de l'objet **connexion**

Code Test correspondent..

```
public class TestConnection {  
public static void main(String args[]) {  
    DBFactory factory;  
  
    factory = new DBFactory("Oracle");  
  
    Connection connection = factory.createConnection();  
  
}  
}
```


La classe Factory correspondante...

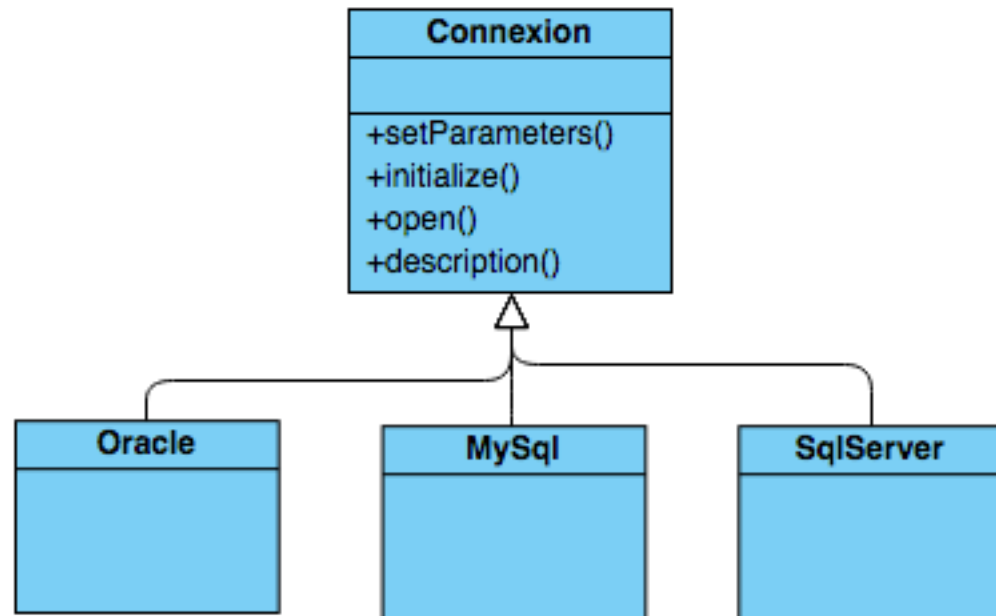
```
public class DBFactory {  
    protected String type;
```

```
    public DBFactory (String t) {  
        type = t; }  
}
```

```
    public Connection createConnection() {  
        if (type.equals("Oracle")){ return new OracleConnection();  
        }  
        else if (type.equals("SQL Server")){  
            return new SqlServerConnection(); }  
        else {  
            return new MySqlConnection();  
        } }  
}
```

Un processus de fabrication

- Connexion con= factory.createConnexion();
- Con.setParameters();
- Con.initialize();
- Con.open();
-
- Ces opérations sont communes à toutes les connexions → Abstraction dans une classe



classe abstraite de connexion

```
public abstract class Connection {
```

```
    public Connection() {}
```

```
    Public void setParameters(String str1,String str2) {.....;}
```

```
    Public void initialize() {.....;}
```

```
    Public void open(){.....;}
```

```
    public String description() {
```

```
        return "Generic"; }
```

```
}
```

```
Class test.....
```

```
DBFactory factory;
```

```
factory = new DBFactory("Oracle");
```

```
Connexion connection = factory.createConnection();
```

```
connection.setParams("username", "Steve");
```

```
connection.setParams("password", "Open the door!!!");
```

```
connection.initialize();
```

```
connection.open();
```

```
Connection.description()
```

```
...
```

Les classes concrètes de connexion

```
public class
OracleConnection extends
Connection {
public OracleConnection() {
}
//.....les autres
méthodes
public String description() {
return "Oracle Connection";
}
}
```

```
public class
SqlServerConnection
extends Connection {
public
SqlServerConnection() {
}
//.....les autres
méthodes
public String description() {
return "SQL Server
Connection"; }
}
```

```
public class
MySQLConnection extends
Connection {
public MySQLConnection() {
}
//.....les autres
méthodes
public String description() {
return "MySQL Connection";
}
}
```

Donc, jusqu'ici, la classe test peut
être....

```
public class TestConnection {  
public static void main(String args[]) {  
    DBFactory factory;  
    factory = new DBFactory("Oracle");  
  
    Connection connection = factory.createConnection();  
  
    System.out.println("You're connecting with " +  
        connection.description());  
}  
}
```

Et Maintenant....

- Jusqu'ici, on a seulement traité le simple factory, car le GoF disent:
- Factory Method design pattern defines an interface for creating an object *but let subclasses **decide*** wich class to instanciate
- Ce principe n'a pas été respecté dans les 2 exemples qu'on a vu dans le cours

Pour respecter ce principe...

- Imaginez que votre entreprise ne veut plus du DBFactory, car maintenant elle préfère une BDD sécurisée: Secure Factory..
- Vous êtes obligé donc de réécrire, modifier la classe DBFactory chaque fois qu'il y a une nouvelle demande
- Pour éviter ces mises à jour répétitifs de la classe DBFactory → créer une classe Factory abstraite, et laisser libre aux développeur de l'adapter à leurs besoins...

Créer la factory abstraite

```
public abstract class ConnectionFactory {  
  public ConnectionFactory() {  
  }  
  
  protected abstract Connection  
  createConnection(String type);  
}
```

Créer la factory concrete

```
public class SecureFactory extends ConnectionFactory {  
  
    public Connection createConnection(String type) {  
  
        if (type.equals("Oracle"))  
            return new SecureOracleConnection();  
  
        else if (type.equals("SQL Server"))  
            return new SecureSqlServerConnection();  
  
        else  
            return new SecureMySQLConnection();  
    }  
}
```

Créer les classes de Connection sécurisée (produits concrets)

```
public class SecureOracleConnection extends Connection {  
public SecureOracleConnection() {  
}  
public String description() {  
return "Oracle secure"; }  
}  
++++  
public class SecureSqlServerConnection extends Connection {  
public SecureSqlServerConnection() {  
}  
public String description() {  
return "SQL Server secure"; }  
}  
  
+++++++  
public class SecureMySQLConnection extends Connection {  
public SecureMySQLConnection() {  
}  
public String description() {  
return "MySQL secure"; }  
}
```

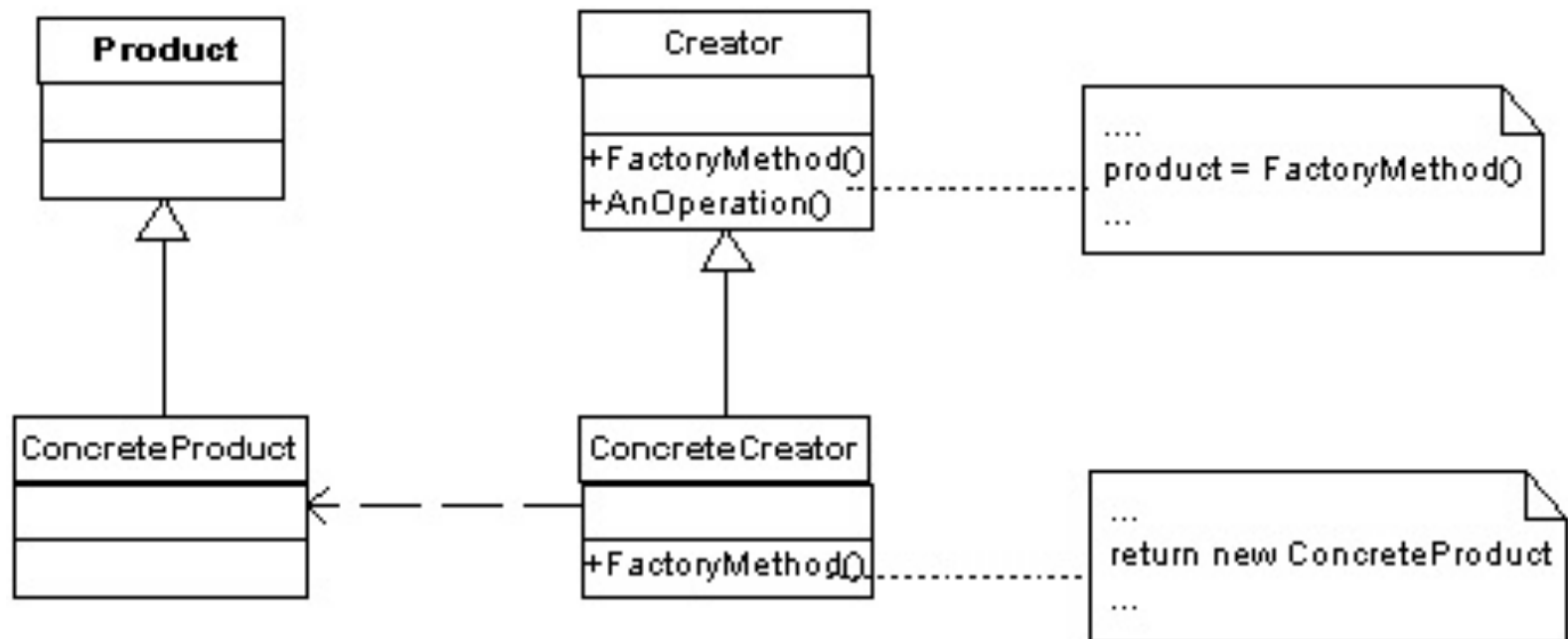
La nouvelle classe de test

```
public class TestFactory {  
public static void main(String args[]) {  
  
    SecureFactory sFactory;  
  
    sFactory = new SecureFactory();  
  
    Connection connection =  
    sFactory.createConnection("Oracle");  
  
    System.out.println("You're connecting with " +  
    connection.description());  
} }
```

Factory method, encore une fois...

- Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- Permet de séparer la création d'objets dérivant d'une classe mère de leur utilisation.
- De ce fait, on a alors la possibilité de créer plusieurs objets issue d'une même classe mère.

Factory method, Diagramme de classe



Factories in java...

Par exemple:

XMLReaderFactory qui permet de créer des objets
XMLReader:

```
try {  
XMLReader reader =  
XMLReaderFactory.createXMLReader(); } catch (SAXException  
e) {System.err.println(e.getMessage());  
}
```

Nous avons vu aussi Factory dans Corba, lors de
l'utilisation de JNDI, ..il y a pas mal d'autres utilisations.

Factories in java...suite

- [java.util.Calendar#getInstance\(\)](#)
- [java.util.ResourceBundle#getBundle\(\)](#)
- [java.text.NumberFormat#getInstance\(\)](#)
- [java.nio.charset.Charset#forName\(\)](#)
- [java.net.URLStreamHandlerFactory#createURLStreamHandler\(String\)](#) (Returns singleton object per protocol)
- [java.util.EnumSet#of\(\)](#)
- [javax.xml.bind.JAXBContext#createMarshaller\(\)](#)
- et plein d'autres...

Abstract Factory, Diagramme de classe

